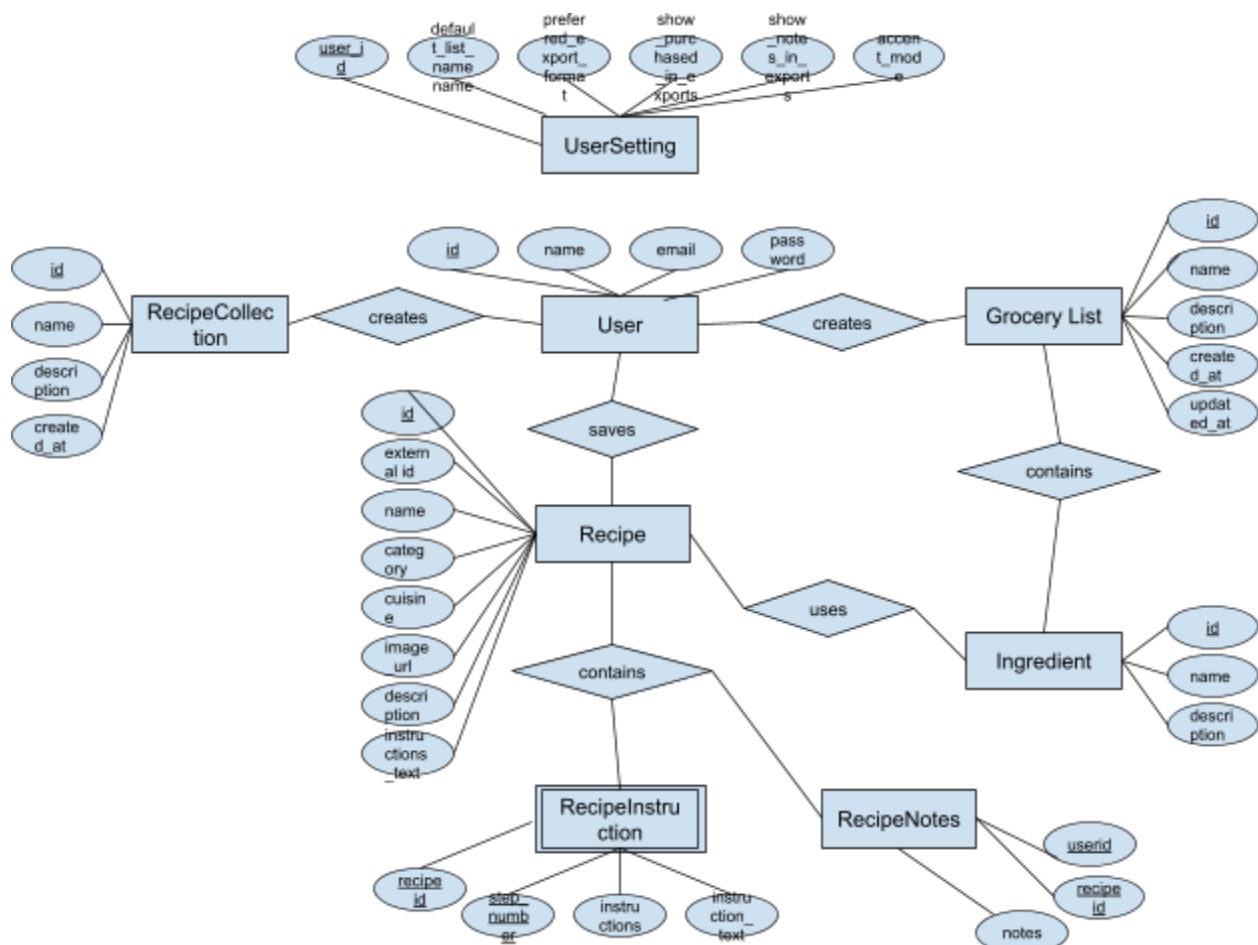


Sia Savkoor - pyk5ux
Suzu Paudel - uwz3vh
Neeha Mikkilineni - gfv5mu

Extra Credit: We received help from Laxmi Ghanate (bnk2zk), Sahiti Palleti (hfd8vv), Max Grant (gbe9wk) when we were facing conflicting schema issues and we helped them with Google Cloud issues. We both learned a lot from collaborating with each other.

1. Database design

- Include a final version of your E-R diagram



- Include a final version of your tables written in schema statements

users(id, name, email, password, role)

recipes(id, external_id, creator_id, name, category, cuisine, image_url, description, instructions_text)

ingredients(id, name, description)

recipe_instructions(recipe_id, step_number, instructions, instruction_text)

uses(recipe_id, ingredient_id, quantity, unit)

grocery_lists(id, creator_id, name, description, created_at, updated_at)

contains(grocery_list_id, ingredient_id, quantity, unit, purchased, note)

saves(user_id, recipe_id, is_favorite, saved_at)

recipe_notes(user_id, recipe_id, note)

recipe_collections(id, user_id, name, description, created_at)

user_settings(user_id, default_list_name, preferred_export_format,
show_purchased_in_exports, show_notes_in_exports, accent_mode)

2. Database programming

- Specify where you host your database

If you do the extra credits (hosting your database and your app on GCP), submit screenshot(s) showing your database on GCP

We host our database on Google Cloud Platform using Cloud SQL for MySQL. Our PantryPal MySQL database is stored in a Cloud SQL instance, which contains the tables for users, recipes, ingredients, saved recipes, grocery lists, recipe instructions, recipe reviews, recipe notes, and user settings.

The screenshot shows the Google Cloud console interface. At the top, there's a search bar and a filter for 'pantrypal'. The main section is titled 'Databases'. Below this, it shows 'All instances > pantrypal-db' and 'MySQL 8.0'. A notification banner states: 'Instance is being updated. This may take a few minutes. While this operation is running, you may continue to view information about the instance.' Below the notification is a '+ Create database' button. A table lists the database instances:

Name ↑	Collation	Character set	Type	
information_schema	utf8mb3_general_ci	utf8mb3	System	⋮
mysql	utf8mb3_general_ci	utf8mb3	System	⋮
pantrypal	utf8mb4_0900_ai_ci	utf8mb4	User	⋮
performance_schema	utf8mb4_0900_ai_ci	utf8mb4	System	⋮
sys	utf8mb4_0900_ai_ci	utf8mb4	System	⋮

On the right side of the console, there is a blue 'Uploads' button.

- Specify where you host your app — the deployment environment needed to deploy and run your project
 - If you do the extra credits (hosting your database and your app on GCP), submit the URL to your app

The PantryPal app is hosted in our GitHub repository and run locally for the final demonstration. The app runs in a local Node.js/npm environment with a React/Vite frontend and an Express/Node backend. The backend connects to our Google Cloud SQL MySQL database using private environment variables in a .env file. To run the project, a user clones the repository, installs dependencies, configures the database environment variables, and starts the frontend and backend with npm.

- Describe and include all instructions / steps needed to deploy and run your project

Note: we may randomly set up, deploy, and run your project on a similar environment. It is important that you provide the description and a complete set of the instructions.

From the project root, first install the root helper package by running `npm install`. Once that completes, install all app dependencies with `npm run install:all`. Configure the backend environment file by creating `.env` in the server directory. Add the following:

```
DB_HOST=35.245.162.21
DB_USER=pantrypal_user
DB_PASSWORD=Password123
DB_NAME=pantrypal
DB_PORT=3306
PORT=3001
```

After everything is set up, start both the frontend and backend together using `npm run dev`. When the development servers are running, open the Vite URL displayed in your terminal to view the application. Use `ctrl c` to terminate

3.
 - Refer to advanced SQL commands you wrote in the previous milestone, clearly discuss how the advanced SQL commands are incorporated in your app, what features of your app use them, and how they reflect the database

We used the following advanced SQL commands in our app:

- 1.

```
DELIMITER $$
```

```
CREATE PROCEDURE GetUserRecipes(IN userId INT)
BEGIN
  SELECT
    r.id,
    r.external_id,
    r.creator_id,
    r.name,
    r.category,
    r.cuisine,
    r.image_url,
    r.description,
    r.instructions_text,
    s.is_favorite,
    s.saved_at
  FROM saves s
  JOIN recipes r ON r.id = s.recipe_id
```

```

WHERE s.user_id = userId
ORDER BY s.saved_at DESC;
END $$

```

DELIMITER ;

The stored procedure GetUserRecipes is used to retrieve the recipes saved by a specific user. This supports the My Recipes feature of the app. When a user views their saved collection, the app needs to return only the recipes connected to that logged-in user. This reflects the database because users and recipes have a many-to-many relationship represented by the saves table, so the procedure joins saves with recipes to retrieve user-specific saved recipes.

2.

DELIMITER \$\$

```

CREATE TRIGGER before_insert_recipe_note
BEFORE INSERT ON recipe_notes
FOR EACH ROW
BEGIN
    IF LENGTH(NEW.note) > 500 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Note cannot exceed 500 characters';
    END IF;
END $$

```

DELIMITER ;

The before_insert_recipe_note trigger is used when a user adds a new personal note to a saved recipe. This supports the Saved Recipe Details page, where users can write notes for recipes they saved. The trigger checks that the note is not longer than 500 characters before inserting it into the recipe_notes table. This reflects the database because recipe_notes stores notes for a specific (user_id, recipe_id) pair, so the trigger protects the validity of that relationship data.

3.

DELIMITER \$\$

```

CREATE TRIGGER before_update_recipe_note
BEFORE UPDATE ON recipe_notes
FOR EACH ROW
BEGIN
    IF LENGTH(NEW.note) > 500 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Note cannot exceed 500 characters';
    END IF;
END $$

```

DELIMITER ;

The before_update_recipe_note trigger is used when a user edits an existing recipe note. This also supports the Saved Recipe Details page. Since users can update notes after creating them, this trigger

makes sure edited notes still follow the 500-character limit. This reflects the database by enforcing the same integrity rule on both new and existing rows in `recipe_notes`, keeping the stored user-recipe note data consistent.

4. Describe the database security at the database level

- Specify whether the security is set for developers or end users
- Discuss how you set up security at the database level (access control)
- Submit the SQL commands you use to limited / set privileges such as

```
GRANT privilege-name ON object-name TO {user-name | PUBLIC  
| role-name}
```

 or

```
REVOKE privilege-name ON object-name FROM {user-name |  
PUBLIC | role-name}
```

```
CREATE ROLE 'app_user';  
CREATE ROLE 'admin_user'; CREATE USER 'app_client'@'localhost' IDENTIFIED BY  
'strongpassword'; CREATE USER 'admin_client'@'localhost' IDENTIFIED BY  
'strongpassword'; GRANT 'app_user' TO 'app_client'@'localhost'; GRANT 'admin_user'  
TO 'admin_client'@'localhost'; SET DEFAULT ROLE 'app_user' TO  
'app_client'@'localhost'; SET DEFAULT ROLE 'admin_user' TO  
'admin_client'@'localhost'; GRANT SELECT, INSERT, UPDATE, DELETE ON  
recipe_app.recipes TO 'app_user'; GRANT SELECT ON recipe_app.ingredients TO  
'app_user'; GRANT SELECT, INSERT, UPDATE, DELETE ON recipe_app.grocery_lists  
TO 'app_user'; GRANT SELECT, INSERT, UPDATE, DELETE ON recipe_app.saves TO  
'app_user'; GRANT ALL PRIVILEGES ON recipe_app.* TO 'admin_user';
```

5. Describe the database security at the application level

- Discuss how database security at the application level is incorporated in your project.
- Submit code snippet(s) to illustrate how security aspect is implemented and to support your discussion.

Database security in PantryPal is enforced at the application level by ensuring that all database interactions are controlled and validated by the backend rather than trusting client input. First, user passwords are never stored in plaintext; instead, they are securely hashed using the Node.js `crypto.scrypt` algorithm with a random salt before being inserted into the database, and verified during login using a safe comparison method. Second, the application replaces the insecure practice of trusting a client-provided user ID with a signed token-based authentication system. When a user logs in or signs up, the server generates a signed token using HMAC, which the client

must include in future requests; this token is verified on every protected route to confirm the user's identity.

```
export function validatePassword(password) {
  if (typeof password !== 'string' || password.length < 8) {
    throw new Error('Password must be at least 8 characters long.')
  }
}

export function isPasswordHash(value) {
  return typeof value === 'string' && value.startsWith(`${PASSWORD_HASH_PREFIX}$`)
}

export async function hashPassword(password) {
  validatePassword(password)

  const salt = crypto.randomBytes(16).toString('hex')
  const derivedKey = await scryptAsync(password, salt, PASSWORD_KEY_LENGTH)

  return `${PASSWORD_HASH_PREFIX}${salt}${Buffer.from(derivedKey).toString('hex')}`
}

export async function verifyPassword(password, storedPassword) {
  if (typeof password !== 'string' || typeof storedPassword !== 'string') {
    return false
  }

  if (!isPasswordHash(storedPassword)) {
    return safeCompare(password, storedPassword)
  }

  const [, salt, expectedHash] = storedPassword.split('$')
  if (!salt || !expectedHash) {
    return false
  }
}
```

```

export function createAuthToken(user) {
  const payload = {
    sub: Number(user.id),
    exp: Date.now() + TOKEN_TTL_MS,
  }
  const encodedPayload = encodeBase64Url(JSON.stringify(payload))
  const signature = crypto
    .createHmac('sha256', tokenSecret)
    .update(encodedPayload)
    .digest('base64url')

  return `${encodedPayload}.${signature}`
}

export function verifyAuthToken(token) {
  const [encodedPayload, signature] = String(token || '').split('.')
  if (!encodedPayload || !signature) {
    throw new Error('Invalid token format.')
  }

  const expectedSignature = crypto
    .createHmac('sha256', tokenSecret)
    .update(encodedPayload)
    .digest('base64url')

  if (!safeCompare(signature, expectedSignature)) {
    throw new Error('Invalid token signature.')
  }

  const payload = JSON.parse(decodeBase64Url(encodedPayload))
  if (!Number.isInteger(payload.sub) || payload.exp <= Date.now()) {
    throw new Error('Token expired or malformed.')
  }

  return { id: payload.sub }
}

```